

pseudo_cl_usage

June 16, 2026

1 jax_healpy.pseudo_cl — differentiable masked-sky power spectra

A pure-JAX, end-to-end **differentiable** pseudo-Cl (MASTER) estimator for the cut sky: apodize a mask -> (optionally) purify the spin-2 B-mode (Smith 2006) -> build the mode-coupling matrix (MCM) -> decouple into unbiased bandpowers. Validated against NaMaster.

Requires 64-bit precision – the spin-2 decoupling solve is ill-conditioned and silently returns all-NaN in float32, so the entry points raise if x64 is off.

Note on B-modes: scalar and spin-2 **EE** decoupling are NaMaster-exact. The purified-**BB** *decoupling* is being upgraded to a dedicated pure-B mode-coupling matrix (see HANDOUT/); the purification + EE machinery shown here is validated.

```
[1]: import jax
jax.config.update('jax_enable_x64', True) # mandatory for pseudo_cl
import jax.numpy as jnp
import numpy as np
import matplotlib.pyplot as plt
import healpy as hp
import jax_healpy as jhp
from jax_healpy import pseudo_cl as pcl

nside, nlb = 64, 16
npix = jhp.nside2npix(nside)
lmax = 3 * nside - 1
print(f'nside={nside} lmax={lmax} npix={npix}')
```

```
nside=64 lmax=191 npix=49152
```

1.1 1. Apodize a mask

apodize reproduces NaMaster's C2 window. We use a smooth polar cap (no corners, SO/SAT-like).

```
[2]: theta, phi = jhp.pix2ang(nside, jnp.arange(npix))
binary = jnp.where(theta < jnp.deg2rad(40.0), 1.0, 0.0) # 40 deg cap
apo = pcl.apodize(binary, aposize_deg=5.0) # differentiable C2
      ↪window
```

```

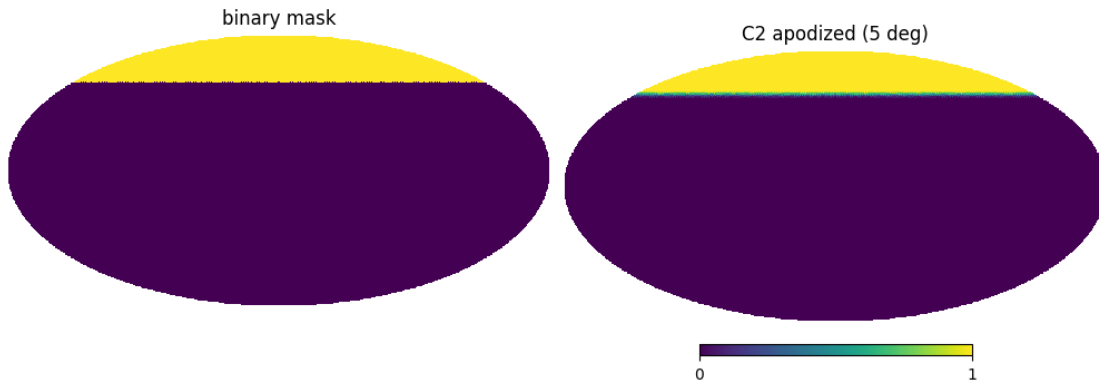
print(f'fsky={float(jnp.mean(binary)):.3f} apo in [{float(apo.min()):.2f}, {float(apo.max()):.2f}]')

fig = plt.figure(figsize=(10, 4))
hp.mollview(np.asarray(binary), sub=121, title='binary mask', cbar=False)
hp.mollview(np.asarray(apo), sub=122, title='C2 apodized (5 deg)')
plt.show()

```

An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Falling back to cpu.

fsky=0.116 apo in [0.00, 1.00]



1.2 2. The apodization *scale* is a differentiable parameter

For the optimizable-apodization research layer, split the (static) grassfire geometry from the differentiable taper: `grassfire_distance` is computed once; `apodize_profile` is differentiable in `theta_star_deg`, so the scale can be tuned by gradient descent.

```

[3]: dist = pcl.grassfire_distance(binary, max_aposize_deg=10.0)  # static niter
    from the max bound
W3, W7 = pcl.apodize_profile(dist, 3.0), pcl.apodize_profile(dist, 7.0)

def edge_weight(theta_deg):
    return jnp.sum(pcl.apodize_profile(dist, theta_deg) ** 2)

g = jax.grad(edge_weight)(5.0)
print(f'd/dtheta* of sum(W^2) at 5 deg = {float(g):.4e} (finite & nonzero ->
    optimizable)')

```

d/dtheta* of sum(W^2) at 5 deg = -1.6080e+02 (finite & nonzero -> optimizable)

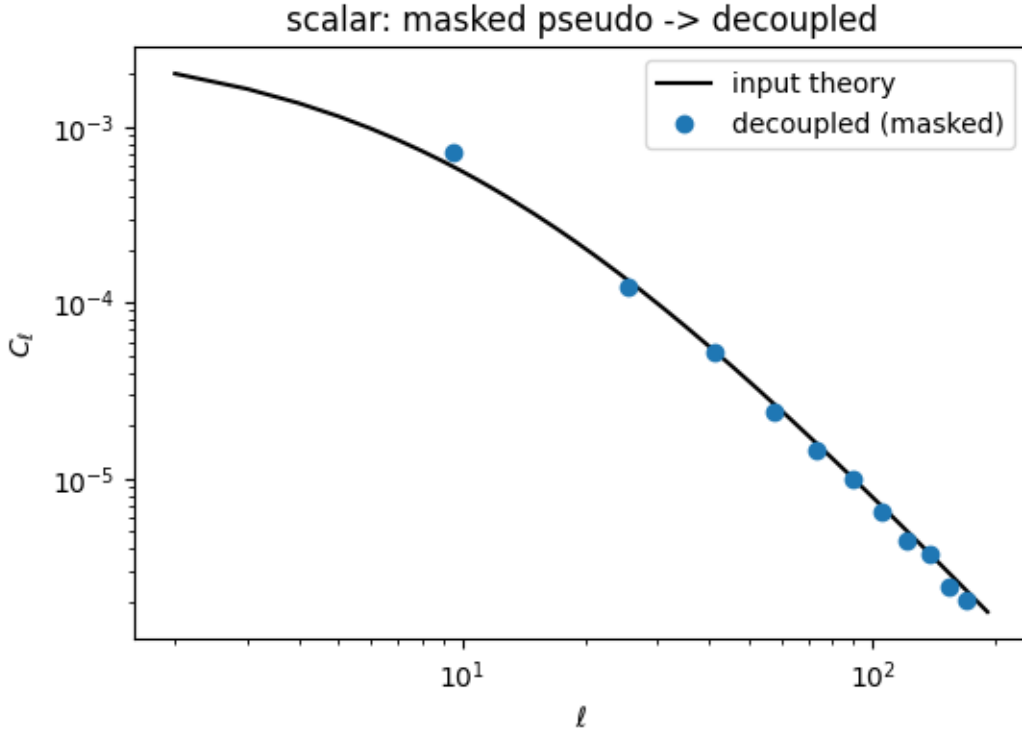
1.3 3. Scalar masked spectrum -> decoupled bandpowers

Pass a mask and `anafast_masked` builds the MCM and returns **decoupled** bandpowers (`ell` = effective bandpower multipoles). This matches NaMaster to `rel < 1e-8`.

```
[4]: ell = np.arange(lmax + 1)
cl_in = np.zeros(lmax + 1); cl_in[2:] = 1.0 / (ell[2:] + 10.0) ** 2.5
m = jhp.synfast(jax.random.key(0), jnp.asarray(cl_in), nside, lmax=lmax,
    ↪pol=False)

ellb, clb = pcl.anafast_masked(m, mask=apo, lmax=lmax, nlb=nlb) # decoupled

plt.figure(figsize=(6, 4))
plt.loglog(ell[2:], cl_in[2:], 'k-', label='input theory')
plt.loglog(np.asarray(ellb), np.asarray(clb), 'o', label='decoupled (masked)')
plt.xlabel(r'$\ell$'); plt.ylabel(r'$C_\ell$'); plt.legend()
plt.title('scalar: masked pseudo -> decoupled'); plt.show()
```



1.4 4. Spin-2 (Q, U) -> EE/BB with B-mode purification

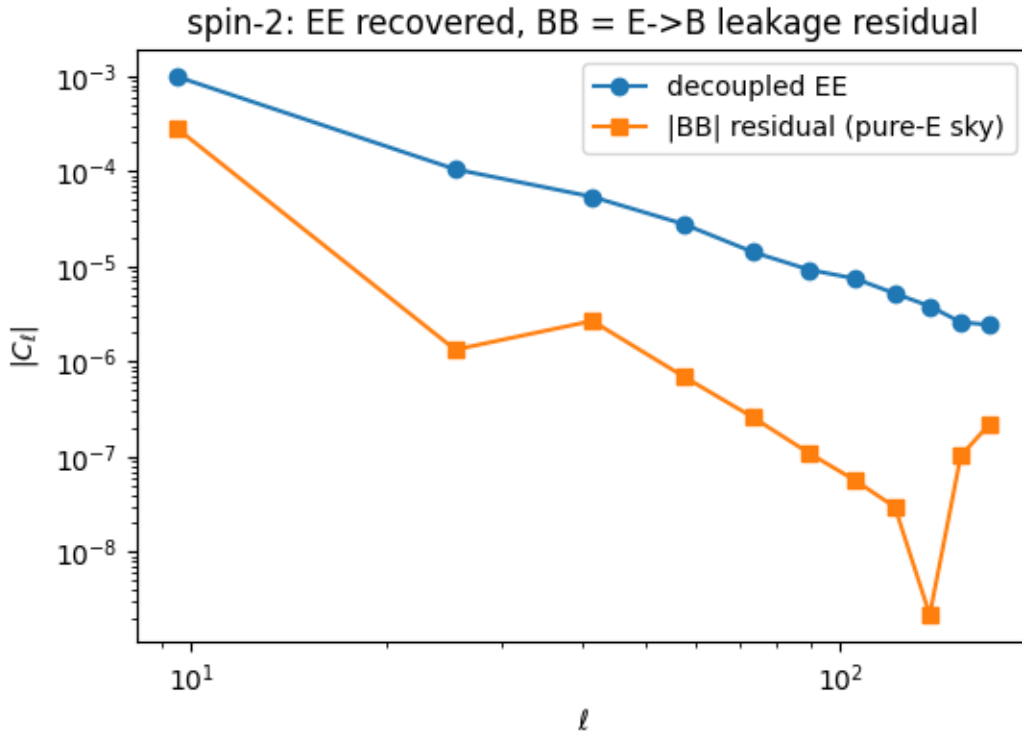
Pass `pol=True` with a (2, npix) (Q, U) map. `purify_b=True` applies the Smith-2006 purification before estimation. Here we feed a **pure-E** sky (true BB = 0) and recover EE; the BB row is the residual E->B leakage.

```
[5]: E = jhp.synalm(jax.random.key(1), jnp.asarray(cl_in), lmax=lmax,
    ↪healpy_ordering=True)
zero = jnp.zeros_like(E)
_, Q, U = jhp.alm2map([zero, E, zero], nside, lmax=lmax, pol=True,
    ↪healpy_ordering=True)
qu = jnp.stack([Q, U], axis=0)

ellb, cl = pcl.anafast_masked(qu, mask=apo, lmax=lmax, nlb=nlb, pol=True,
    ↪purify_b=True) # (EE, EB, BB)
print('returned spectra shape:', cl.shape, '= (EE, EB, BB)')

plt.figure(figsize=(6, 4))
plt.loglog(np.asarray(ellb), np.abs(np.asarray(cl[0])), 'o-', label='decoupled_
    ↪EE')
plt.loglog(np.asarray(ellb), np.abs(np.asarray(cl[2])), 's-', label='|BB|_
    ↪residual (pure-E sky)')
plt.xlabel(r'$\ell$'); plt.ylabel(r'$|C_\ell|$'); plt.legend()
plt.title('spin-2: EE recovered, BB = E->B leakage residual'); plt.show()
```

returned spectra shape: (3, 11) = (EE, EB, BB)



1.5 5. Everything is differentiable

The whole estimator is a differentiable JAX function – gradients flow through apodization, the MCM, purification and decoupling. This is what enables gradient-based optimization of the apodization and end-to-end component-separation pipelines.

```
[6]: def loss(x):
      _, cc = pcl.anafast_masked(x, mask=apo, lmax=lmax, nlb=nlb)
      return jnp.sum(cc)

grad = jax.grad(loss)(m)
print('grad through (map -> mask -> MCM -> decoupled C1):',
      'finite=', bool(jnp.all(jnp.isfinite(grad))),
      ' nonzero=', float(jnp.sum(jnp.abs(grad))) > 0)
```

grad through (map -> mask -> MCM -> decoupled C1): finite= True nonzero= True

1.6 6. Precompute the MCM once, reuse it

The MCM depends only on the mask, so build it once with `compute_mcm` and pass it back via `mcm=` (e.g. frozen for many maps, or inside an inference loop).

```
[7]: mcm = pcl.compute_mcm(apo, lmax=lmax, nlb=nlb, pol=False) # eqx.Module, ↵
      ↪ reusable / differentiable
_, a = pcl.anafast_masked(m, mask=apo, lmax=lmax, nlb=nlb)
_, b = pcl.anafast_masked(m, mask=apo, lmax=lmax, nlb=nlb, mcm=mcm)
print('precomputed MCM gives identical result:', bool(np.allclose(np.
      ↪ asarray(a), np.asarray(b))))
```

precomputed MCM gives identical result: True